

College Finance Bot

Team Members: Ashwin Chhawaniya, Arnav Singh, Akshat Goyal
Symbiosis Artificial Intelligence Institute
Full Code Manual & Technical Documentation

A comprehensive Python-based personal finance management system with MySQL database integration, featuring 8 powerful financial tools for college students and young professionals.

System Architecture & Technology Stack

Core Technologies

Built on Python 3.x with MySQL database backend, the application leverages three critical libraries:

- **mysql-connector-python**: Official Oracle driver for database operations
- **matplotlib**: Creates interactive financial projection graphs
- **forex-python**: Fetches real-time currency exchange rates

Database Configuration

```
DB_CONFIG = {  
    "host": "localhost",  
    "user": "root",  
    "password": "javamx",  
    "database": "college_finance_db"  
}
```

The system automatically creates the database and all 8 tables on first run, ensuring seamless deployment.

Database Schema: 8 Interconnected Tables

users

Core authentication table storing unique usernames and passwords with auto-incrementing IDs

entries

Financial ledger tracking all income/expense transactions with timestamps and tags

monthly_budget

Budget plans with three custom priorities and percentage allocations

bill_splits

Historical record of bill splitting calculations with per-person shares

tax_calculations

Tax computation history with income, tax due, and effective rates

compound_interest_records

Investment projections with principal, contributions, and final amounts

currency_conversions

Currency exchange logs with from/to currencies and converted amounts

savings_records

Savings plan projections with monthly contributions and final balances

All tables link to the **users** table via **user_id** foreign key, ensuring complete data privacy and user-specific record isolation.

Feature 1: Monthly Budget Planner

How It Works

Users define a total monthly budget and allocate it across three custom priorities plus miscellaneous expenses. The system validates percentages don't exceed 100% and calculates remaining allocation automatically.

Database Storage: Each budget plan is saved to the **monthly_budget** table with month identifier, total amount, priority names, and percentage breakdowns.

Code Implementation

```
def monthly_budget_planner(user_id):
    budget = float(input("Total budget: "))
    p1 = input("Priority 1 name: ")
    p1_pct = int(input("Priority 1 %: "))
    p2 = input("Priority 2 name: ")
    p2_pct = int(input("Priority 2 %: "))
    p3 = input("Priority 3 name: ")
    p3_pct = int(input("Priority 3 %: "))

    misc_pct = 100 - (p1_pct + p2_pct + p3_pct)
    month = datetime.datetime.now().strftime("%Y-%m")

    # Insert into monthly_budget table
    cursor.execute("""INSERT INTO monthly_budget
VALUES (NULL, %s, %s, %s, %s, %s, ...)""",
(user_id, month, budget, p1, p1_pct, ...))
```

Features 2 & 3: Bill Splitter and Tax Calculator

1

Bill Splitter

Divides total bill amount equally among specified number of people.
Validates positive integer input and stores calculation history.

```
def bill_splitter(user_id):  
    bill = float(input("Total bill: "))  
    people = int(input("Number of people: "))  
    share = bill / people  
  
    cursor.execute("""INSERT INTO bill_splits  
        VALUES (NULL, %s, %s, %s, %s, NOW())""",  
        (user_id, bill, people, share))  
  
    print(f"Per person: ₹{share:.2f}")
```

2

Tax Calculator

Applies progressive tax slabs to calculate total tax due and effective rate. Uses hard-coded Indian tax system with multiple brackets.

```
def tax_calculator(user_id):  
    income = float(input("Annual income: "))  
    slabs = [(250000, 0), (500000, 0.05),  
        (1000000, 0.20), (float('inf'), 0.30)]  
  
    tax = 0  
    for limit, rate in slabs:  
        taxable = min(remaining, limit - prev)  
        tax += taxable * rate  
  
    effective_rate = (tax / income) * 100
```

Feature 4: Compound Interest Calculator

Advanced Investment Projection with Visualization

This sophisticated tool models investment growth under three scenarios (low, expected, high) based on user-defined variance. It handles different compounding frequencies and generates interactive matplotlib graphs showing projected growth over time.

```
def compound_interest(user_id):
    principal = float(input("Initial investment: "))
    monthly_contrib = float(input("Monthly contribution: "))
    years = float(input("Duration (years): "))
    annual_rate = float(input("Annual rate %: ")) / 100
    variance = float(input("Variance %: ")) / 100
    frequency = input("Frequency (annual/quarterly/monthly): ")

    rates = {
        'low': annual_rate - variance,
        'mid': annual_rate,
        'high': annual_rate + variance
    }

    for scenario, rate in rates.items():
        balance = principal
        balances = [balance]
        for month in range(int(years * 12)):
            balance += monthly_contrib
            if frequency == 'monthly':
                balance *= (1 + rate/12)
            balances.append(balance)

    plt.plot(balances, label=scenario)

plt.show()
```

Features 5 & 6: Currency Converter and Savings Calculator

Currency Converter

Offers choice between **live API rates** (forex-python) or **fixed offline rates**. Supports 10 global currencies including USD, EUR, GBP, JPY, and more.

```
def currency_converter(user_id):
    c = CurrencyRates()
    amount = float(input("Amount: "))
    from_curr = input("From (USD/EUR/INR): ")
    to_curr = input("To (USD/EUR/INR): ")

    use_fixed = input("Use fixed rates? (y/n): ")

    if use_fixed == 'y':
        # Use offline conversion_rates dict
        converted = convert_via_inr(amount,
                                   from_curr,
                                   to_curr)
    else:

        converted = c.convert(from_curr,
                              to_curr,
                              amount)

    print(f"{amount} {from_curr} = "
          f"{converted:.2f} {to_curr}")
```

Savings Calculator

Projects future balance based on monthly savings and annual return rate. Generates growth visualization using matplotlib.

```
def savings_calculator(user_id):
    monthly = float(input("Monthly saving: "))
    years = float(input("Duration (years): "))
    annual_return = float(input("Annual return %: "))

    monthly_rate = annual_return / 12 / 100
    months = int(years * 12)

    balance = 0
    balances = []

    for month in range(months):
        balance = balance * (1 + monthly_rate)
        balance += monthly
        balances.append(balance)

    plt.plot(balances)
    plt.title('Savings Growth Projection')
    plt.show()

    print(f"Final balance: ₹{balance:.2f}")
```

Feature 7: Record Income/Expense - The Core Ledger

01

Entry Type Selection

User chooses between income ('i') or expense ('e') with input validation

03

Database Insertion

Records transaction in entries table with timestamp and user_id

02

Amount & Tag Input

Validates positive amount; income uses predefined tags, expenses allow custom tags

04

Financial Summary

Calculates and displays updated total income, expenses, and current balance

```
def record_entry(user_id):
    entry_type = input("Type (i=income, e=expense): ").lower()
    amount = float(input("Amount: "))

    if entry_type == 'i':
        print("Income tags: Salary, Freelance, Gift, Other")
        tag = input("Select tag: ")
    else:
        tag = input("Expense tag (e.g., Food, Rent): ")
    entry_time = datetime.datetime.now()
    cursor.execute("""INSERT INTO entries
VALUES (NULL, %s, %s, %s, %s, %s)""",
    (user_id, entry_type, amount, tag, entry_time))
    total_income, total_expense, balance = get_user_financial_summary(user_id)

    print(f"\n{'='*50}")
    print(f"Entry recorded: {entry_type.upper()} of ₹{amount:.2f}")
    print(f"Total Income: ₹{total_income:.2f}")
    print(f"Total Expenses: ₹{total_expense:.2f}")
    print(f"Current Balance: ₹{balance:.2f}")
    print(f"\n{'='*50}")
```


User Authentication & Application Flow

1. Database Initialization

The application starts by calling `create_all_tables()` to ensure the database structure is set up correctly for storing financial data.

2. User Login/Registration

Users are presented with options to either register as a new user or log in if they already have an account. This loop ensures secure access before proceeding.

3. Main Menu & Feature Access

Once authenticated, the user enters the main application loop. The `main_menu()` function displays available tools like the budget planner or bill splitter, and the application executes the chosen feature.

4. Application Exit

Users can exit the application from the main menu. Upon choosing to quit, a goodbye message is displayed, and the program terminates gracefully.

```
def main():
    create_all_tables() # Initialize database

    # Login/Registration loop
    while True:
        choice = input("New user or login? (new/login): ")
        if choice == 'new':
            register_user()
        elif choice == 'login':
            user_id = login_user()
            if user_id:
                break

    # Main feature loop
    while True:
        choice = main_menu()
        if choice == '1':
            monthly_budget_planner(user_id)
        elif choice == '2':
            bill_splitter(user_id)
        # ... other features ...
        elif choice == '8':
            print("Goodbye! Have a great day!")
            break
```

Installation & Execution Guide



Install MySQL Server

Download MySQL Community Server or XAMPP and ensure server is running on localhost



Install Python 3.6+

Download from python.org and verify installation with `python --version`



Install Required Libraries

Run: `pip install mysql-connector-python matplotlib forex-python`



Configure Database Credentials

Edit DB_CONFIG dictionary with your MySQL username and password



Execute Application

Run: `python Finance_Bot_FMM.py` from terminal in script directory

❏ **First Run:** The application automatically creates the college_finance_db database and all 8 tables. Subsequent runs connect to existing database, preserving all user data and transaction history.